Tesla AI

Comprehensive Self-Study & Interview Preparation Plan

Foundation Models • 3D Computer Vision • Vision FM (Optimus) • ML Inference Optimization • Data Scaling

Short-term (3 months) + Long-term (12 months) + Deep Interview Prep

1. Overview — The 5 Target Roles

These five open roles sit at the intersection of deep learning research, production systems, and real-world robotics. They share a large common foundation but each demands a distinct area of depth. Understanding the overlap — and the gaps — is the first step to an efficient study plan.

Role	Core domain	Unique skill cluster
Foundation Models	Multimodal LLMs	Video transformers, imitation learning, generative models (diffusion/AR), self-supervised pretraining at scale
3D Computer Vision	Autonomous perception	NeRF, 3D Gaussian Splatting, SLAM, SfM, multi-view geometry, depth & pose estimation, dense reconstruction
Vision FM — Optimus	Embodied AI / robotics	Vision-language-action (VLA) models, real-time robot perception, dexterous manipulation policies
ML Inference Opt.	On-vehicle deployment	QAT/PTQ quantization, structured pruning, distillation, CUDA/Triton kernels, ASIC codesign, edge NPU
Data Scaling	Data engine	Data mixture strategies, active learning, auto-labeling pipelines, pre/post-training data quality, eval design

1.1 Universal Skill Priorities

The skills below appear as explicit requirements in all five job descriptions. These are the highest-leverage items to master before anything role-specific.

Deep learning theory: loss functions, optimizers, regularization	*Must-have*
Self-supervised & semi-supervised learning	*4 of 5 roles*
Computer vision: detection, segmentation, tracking	*4 of 5 roles*
Strong Python software engineering practices	*Must-have*
GPU training: mixed precision (bf16/fp16), gradient checkpointing	*Must-have*
Evaluation design & metric engineering	*All data-facing roles*
Working with very large datasets (TB–PB scale)	*3 of 5 roles*

2. Short-Term Plan — 3 Months

Goal: build a rock-solid, unified foundation covering all skills shared by the five roles, and ship at least one tangible project. By the end of month 3 you should be able to answer any core DL theory question cold and have something on GitHub to point to.

Month 1 Deep Learning Fundamentals + PyTorch Mastery | *2–3 hrs/day*

Loss functions, optimizers & regularization — from scratch

Tesla interviews go well below the API surface. You need to know why things work, not just how to call them.

- Derive cross-entropy loss and its gradient w.r.t. logits.
- Implement focal loss; understand why it outperforms CE for class-imbalanced detection.
- Implement contrastive loss (NT-Xent) and triplet loss. Know when to use each.
- Implement SGD with momentum, then Adam, then AdamW from scratch in NumPy.
- Understand the decoupled weight decay distinction between Adam and AdamW.
- Layer norm, batch norm, RMS norm: derive their forward and backward passes.
- Dropout: derive expected value and why inference uses no dropout but a scale factor.

Resources: *Karpathy's Neural Nets: Zero to Hero* • *PyTorch docs — torch.nn.functional*

Transformer architecture — implement everything from scratch

Do not stop at the conceptual level. Implement a transformer end-to-end.

- Implement scaled dot-product attention. Derive why we scale by 1/sqrt(d_k).
- Implement multi-head attention with KV cache for autoregressive inference.
- Build a GPT-style decoder from scratch and train it on a small corpus (Shakespeare).
- Implement a Vision Transformer (ViT) — patch embedding, positional encoding, CLS token.
- Understand RoPE positional encoding and why modern LLMs prefer it over learned positions.
- Walk through KV cache memory layout and why it dominates inference memory at long context.

Resources: *nanoGPT (Karpathy)* • *Attention Is All You Need* • *ViT paper (Dosovitskiy et al.)*

Self-supervised & contrastive learning — the backbone of Tesla's models

FSD and Optimus pretraining rely heavily on SSL. You must know these methods deeply.

- SimCLR: data augmentation strategy, NT-Xent loss, importance of large batch size.
- DINO: self-distillation without labels, centering trick, momentum encoder, multi-crop.
- CLIP: contrastive image-text pretraining, zero-shot transfer, temperature scaling.
- MAE: masked autoencoding, why high masking ratio (75%) works, decoder design.
- DINOv2: curated data, distillation head, why it outperforms supervised pretraining.
- Know which method to choose for a given downstream task and data availability.

Resources: *DINO paper* • *MAE paper* • *CLIP paper* • *DINOv2 paper*

Distributed training fundamentals

Tesla trains on thousands of H100s. You must be fluent in distributed patterns.

- Data parallelism (DDP): gradient synchronization via all-reduce. Understand ring-allreduce.
- Fully Sharded Data Parallel (FSDP): shard parameters, gradients, and optimizer states.
- ZeRO stages 1/2/3: understand what each shards and the memory/communication tradeoff.
- Gradient checkpointing: recompute activations vs store them — when is each worth it?
- Mixed precision (bf16/fp16): loss scaling, master weights, numerical stability considerations.
- Pipeline parallelism and tensor parallelism: when DDP alone is not enough.

Resources: *PyTorch FSDP tutorial* • *Megatron-LM paper* • *Lillian Weng's blog*

Month 2 Computer Vision Depth + 3D Fundamentals | *2–3 hrs/day*

Object detection, segmentation & tracking — production-grade

Detection and segmentation are used in every Tesla perception pipeline.

- DETR: bipartite matching loss, set prediction, why no NMS. Implement the matching loss.
- DINO-DETR (not to be confused with self-supervised DINO): deformable attention, DAB queries.
- SAM: promptable segmentation, IoU prediction head, ambiguity handling.
- Panoptic segmentation: thing vs stuff, how mask and semantic heads combine.
- Multi-object tracking: SORT (Kalman + Hungarian), ByteTrack, OC-SORT. Know their failure modes.
- Streaming perception: latency-accuracy tradeoff, how to make a detector causal.

Resources: *DETR paper* • *DINO-DETR paper* • *SAM paper* • *ByteTrack paper*

Camera geometry & multi-view — the math behind FSD

Every Tesla camera-based system depends on this. It is non-negotiable for the 3D CV role and strongly recommended for all others.

- Pinhole camera model: intrinsics matrix K, derivation from similar triangles.
- Extrinsics: rotation (SO(3)/quaternions) and translation. Camera-to-world and inverse.
- Image formation: projection, distortion (radial/tangential), undistortion.
- Epipolar geometry: essential matrix E, fundamental matrix F, derivation and properties.
- Stereo vision: disparity, triangulation, depth from stereo. Implement a basic stereo matcher.
- Multi-view stereo: cost volumes, plane sweep stereo, MVSNet architecture.
- Implement camera projection, unprojection, and coordinate frame transforms in PyTorch.

Resources: *Multiple View Geometry — Hartley & Zisserman* • *OpenCV camera calibration docs*

Monocular depth, optical flow & pose estimation

Core perception outputs for self-driving. Understanding the self-supervised training of these is especially important.

- Monocular depth: DepthAnything v2, ZoeDepth. Self-supervised depth via photometric loss.
- Depth scale ambiguity: understand why monocular depth is scale-ambiguous and how to resolve it.
- Optical flow: RAFT architecture (correlation volume, GRU updates). FlowFormer.
- Ego-motion estimation: 5-point algorithm, PnP, RANSAC. Understand DLT for homography.
- 6-DoF pose estimation: PnP, EPnP, iterative refinement. FoundPose / FoundationPose.

Resources: *DepthAnything v2* • *RAFT paper* • *FoundPose paper*

Neural radiance fields (NeRF) + 3D Gaussian Splatting

Both methods are explicitly named in the 3D CV JD and are used for Tesla's simulation and data generation pipelines.

- NeRF: volume rendering integral, quadrature sampling, positional encoding, coarse-to-fine.
- Instant-NGP: hash encoding — why it achieves 1000× speedup over vanilla NeRF.
- Nerfacto (nerfstudio): what a practical, fast NeRF pipeline looks like end-to-end.
- 3D Gaussian Splatting: Gaussian parameters (position, covariance, opacity, SH coefficients).
- 3DGS rasterization: tile-based differentiable splat rendering, alpha compositing.
- Comparison: NeRF vs 3DGS — training time, rendering speed, memory, editability.
- Applications: novel view synthesis for data augmentation, simulation, world model evaluation.

Resources: *Original NeRF paper* • *Instant-NGP paper* • *3DGS paper* • *nerf.studio*

SLAM & Structure from Motion

Needed for the 3D CV role and useful context for all spatial AI work at Tesla.

- Classic SLAM loop: front-end (tracking), back-end (optimization), loop closure.
- ORB-SLAM3: ORB features, map points, keyframe selection, pose graph optimization.
- DROID-SLAM: dense bundle adjustment via GRU-based flow updates — understand its design.
- COLMAP: incremental SfM pipeline — feature extraction, matching, triangulation, BA.
- Visual-Inertial SLAM: IMU pre-integration, tight coupling, why IMU helps scale recovery.

Resources: *ORB-SLAM3 paper* • *DROID-SLAM paper* • *COLMAP docs*

Month 3 Projects + Targeted Role Prep | *Build • Showcase • Practice*

Project 1 — Mini data engine (data scaling role signal)

Design and implement an end-to-end auto-labeling + training loop.

- Choose a dataset: BDD100K, nuScenes, or KITTI.
- Stage 1 — Auto-label: use SAM + GroundingDINO to generate pseudo-labels at scale.
- Stage 2 — Filter: implement a quality score (confidence threshold + IoU with known GT).
- Stage 3 — Train: finetune a detection model on the pseudo-labeled data.
- Stage 4 — Eval: measure mAP before/after filtering. Track label noise vs data quantity tradeoff.
- Document the pipeline on GitHub with a detailed README and loss curves.

Project 2 — Quantize and benchmark a ViT (inference opt. role signal)

Demonstrates awareness of the full deployment stack.

- Take a pretrained ViT-B from HuggingFace. Establish baseline FP32 latency and accuracy.
- Apply post-training quantization (PTQ) to INT8 using torch.quantization. Measure degradation.
- Apply QAT: finetune with fake-quantization nodes inserted in forward pass.
- Compare PTQ vs QAT: accuracy vs calibration data requirement vs training time.
- Profile with torch.profiler: identify the top-3 slowest ops and memory bottlenecks.
- Export to ONNX, run with ONNXRuntime, compare throughput vs PyTorch eager.

Resources: *torch.quantization docs* • *HuggingFace Optimum* • *torch.profiler tutorial*

LeetCode — daily coding practice

Tesla runs standard SWE coding interviews. Do not neglect this regardless of how strong your ML background is.

- Arrays & two-pointer: sliding window, prefix sums, intervals.
- Trees & graphs: DFS, BFS, topological sort, union-find.
- Dynamic programming: 1D/2D DP, memoization vs tabulation.
- Heaps & priority queues: top-K problems, streaming median.
- Strings: KMP, trie, rolling hash.
- Target: 2 medium problems per day. Do at least 5 hard problems per week by month 3.

ML system design — whiteboard practice

Practice designing full ML systems end-to-end on a whiteboard (or Excalidraw). This is a separate skill from building them.

- Use the framework: data → model → training → eval → deployment → monitoring.
- Practice with a partner or record yourself. Clarity of communication is graded, not just correctness.
- Do at least 3 full mock system design sessions: (1) occupancy network, (2) data engine, (3) on-vehicle inference stack.

Resources: *Chip Huyen's blog* • *Designing ML Systems (Huyen, O'Reilly)*

3. Long-Term Plan — 12 Months

Goal: move from solid foundations to research-level depth across all five role areas. By month 12 you should be able to walk into any Tesla AI team room and speak fluently about their systems, publish a replication or extension of a recent paper, and demonstrate full ML vertical ownership through your portfolio.

Q1 (months 1–3) Foundation — complete short-term plan

Complete all short-term plan modules

Non-negotiable before advancing. Every module in months 1–3 is a prerequisite for Q2 and beyond. Do not skip or skim.

Q2 (months 4–6) Advanced Architectures + Scaling Laws

Large multimodal models (LMMs) — full pipeline understanding

Tesla's FSD v12 is an end-to-end model. You need to understand how vision and language components are composed at scale.

- LLaVA: visual instruction tuning, projector MLP connecting CLIP encoder to Vicuna LLM.
- Flamingo: cross-attention gated layers, perceiver resampler for variable-length visual tokens.
- InternVL / Qwen-VL: production-grade multimodal models with strong OCR and spatial reasoning.
- PaliGemma: lightweight VLM suitable for robotics — good Optimus-relevant study.
- Train a small VLM: connect a frozen CLIP ViT-B to a 1B LLM using a 2-layer MLP projector. Finetune on LLaVA-Instruct data.
- Understand visual token compression: why sending 256 tokens per image is expensive and how resamplers/poolers help.

Resources: *LLaVA paper* • *Flamingo paper* • *PaliGemma report*

Video transformers & temporal modeling

Driving is inherently a video problem. Temporal reasoning separates weak candidates from strong ones.

- VideoMAE: tube masking (spatial+temporal patches), why 90% masking ratio is optimal for video.
- Video Swin Transformer: 3D shifted window attention, locality inductive bias.
- GROOT / Video-DINO: self-supervised video pretraining with multi-crop and temporal consistency.
- Temporal attention patterns: full 3D attention vs factored spatial+temporal vs causal variants.
- Efficient video training: gradient checkpointing across frames, frame subsampling strategies.
- Build a video classifier on UCF-101 or Kinetics-400 using a pretrained VideoMAE backbone.

Resources: *VideoMAE paper* • *Video Swin Transformer* • *TimeSformer paper*

Scaling laws — the theory behind Tesla's data decisions

The data scaling role specifically requires reasoning about compute-optimal training. This is also tested in foundation model interviews.

- Kaplan et al. (2020): power-law relationships between loss, compute, parameters, and data.
- Chinchilla (Hoffmann et al., 2022): corrects Kaplan — for fixed compute, optimal $N \propto C^{0.5}$, $D \propto C^{0.5}$.
- Implications: most models are over-parameterized and under-trained relative to compute budget.
- Data quality vs quantity: when does filtering a dataset 10× outweigh adding 10× more raw data?
- Pre-training vs post-training (SFT/RLHF) data quality: different distributions and requirements.
- Understand how these laws inform Tesla's decision of how much driving data to collect vs how large to make the model.

Resources: *Chinchilla paper* • *Scaling Laws paper (Kaplan et al.)* • *Beyond Chinchilla Optimal paper*

3D scene understanding — neural + classical

Required for the 3D CV role and highly relevant for FSD occupancy and mapping systems.

- BEV perception: BEVFusion, BEVDet, BEVFormer — lifting camera features to bird's-eye-view.
- Occupancy prediction: TPVFormer, Occ3D — voxel-based occupancy from camera-only inputs.
- Neural scene reconstruction: MonoSDF, RegNeRF — fitting NeRFs under sparse views.
- Dense reconstruction: TSDF Fusion, Open3D, large-scale point cloud processing.
- Implement BEV feature lifting (LSS / Lift-Splat-Shoot) from scratch in PyTorch.

Resources: *BEVFusion paper* • *Lift-Splat-Shoot paper* • *OccNet paper*

Q3 (months 7–9) Inference Optimization + Embodied AI

Model compression — full stack from theory to profiling

The inference optimization role requires you to go from a large model to a deployed, fast model with minimal accuracy loss.

- Post-training quantization (PTQ): calibrate activation ranges with representative data, GPTQ algorithm for weight-only INT4.
- Quantization-aware training (QAT): insert fake-quantization (FQ) nodes, straight-through estimator for gradients.
- INT4 weight + INT8 activation: understand why INT4 weights with INT8 activations often works better than INT4/INT4.
- Structured pruning: prune entire attention heads or FFN neurons. Retrain with distillation loss.
- Unstructured pruning: magnitude pruning, SparseGPT (second-order weight updates, layer-wise).
- Knowledge distillation: soft labels (Hinton), feature distillation (FitNets, CRD), contrastive distillation.
- Profile a compressed model: use NVIDIA Nsight, torch.profiler, or Perfetto. Identify roofline position.

Resources: *GPTQ paper* • *SparseGPT paper* • *AWQ paper* • *Hinton KD paper*

CUDA programming + custom Triton kernels

Writing custom GPU kernels separates candidates who understand hardware from those who don't. This level of depth is what Tesla's inference optimization team looks for.

- GPU memory hierarchy: HBM (global), L2, L1/shared, registers. Understand bandwidth vs compute limits.
- CUDA thread hierarchy: grids, blocks, warps, threads. Coalesced memory access patterns.
- Write a tiled matrix multiplication in CUDA. Profile with Nsight to see occupancy and stall reasons.
- Triton: write a fused softmax kernel, a fused layer-norm kernel, and a flash-attention forward pass.
- FlashAttention-2: tiling strategy, recomputation in backward, why it's memory-bandwidth-bound not compute-bound.
- CUDA graphs: capture a static computation graph for zero-overhead kernel launches.
- TensorRT: export a PyTorch model, run TensorRT optimization, measure speedup on inference.

Resources: *CUDA Programming Guide* • *OpenAI Triton tutorials* • *FlashAttention-2 paper* • *CUDA by Example (Sanders)*

Imitation learning + robot policy architectures

Core to the Optimus roles. Tesla Optimus uses learned policies trained on human demonstration data.

- Behavior cloning (BC): supervised learning on (state, action) pairs. Understand distributional shift / compounding error.
- DAgger: online data collection to reduce covariate shift. Interactive imitation learning.
- GAIL: generative adversarial imitation learning — discriminator distinguishes expert from learner rollouts.
- ACT (Action Chunking with Transformers): predict action chunks, temporal ensembling, why chunking reduces error accumulation.
- Diffusion Policy: model action distribution as a denoising diffusion process. DDPM vs DDIM inference.
- Train an ACT policy in simulation (LIBERO, RoboSuite, or MuJoCo). Evaluate on held-out tasks.

Resources: *ACT paper* • *Diffusion Policy paper* • *GAIL paper* • *LIBERO benchmark*

World models & generative data augmentation

Tesla's approach to rare-scenario coverage relies on generative models. Understanding this is a strong differentiator.

- GAIA-1: autoregressive world model that predicts future video frames of driving given actions.
- DriveDreamer / WoVogen: diffusion-based driving world models, controllable scene generation.
- UniSim: universal neural simulator for sensor data — generate LiDAR+camera from scene description.
- How to use world models for data augmentation: inject rare events (pedestrian occlusion, rain) into synthetic rollouts.
- Evaluation of synthetic data: does adding X synthetic samples improve eval on real-world held-out data?

Resources: *GAIA-1 paper* • *DriveDreamer paper* • *UniSim paper*

Q4 (months 10–12) Research-Level Work + Interview Blitz

Capstone project — open-source, end-to-end, production-grade

This is your portfolio centerpiece. It should demonstrate the full ML vertical ownership Tesla cares about.

- Choose a scope: (a) camera-only BEV occupancy network trained on nuScenes, or (b) VLA policy for robot manipulation trained in simulation.
- Data stage: document your data curation pipeline, filtering decisions, and label statistics.
- Model stage: design your architecture from first principles. Justify every design choice.
- Training stage: use DDP/FSDP, mixed precision, gradient checkpointing. Log everything with W&B.
- Eval stage: build a rigorous eval suite with multiple metrics, per-scenario breakdowns, and failure analysis.
- Deployment stage: quantize to INT8, export to ONNX, benchmark latency on a consumer GPU.
- Open-source: clean code, detailed README with architecture diagram, training curves, and results table.

Study Tesla AI's published work inside-out

You should be able to discuss their specific architecture choices fluently in interviews.

- HydraNet: multi-task head sharing — understand the tradeoffs between task-specific and shared representations.
- FSD v12 end-to-end: the shift from modular to end-to-end neural driving — what changed and why.
- Tesla AI Day 2021: occupancy networks, vector space, the data labeling pipeline at scale.
- Tesla AI Day 2022: Dojo supercomputer, ExaPOD architecture, virtual environments for training.
- Optimus updates: manipulation demos, perception architecture for hands-on manipulation tasks.

Resources: *Tesla AI Day 2021 (YouTube)* • *Tesla AI Day 2022 (YouTube)* • *Tesla technical blog*

Mock interview blitz — minimum 15 sessions

The gap between knowing content and performing under pressure is large. Close it deliberately.

- 5 coding sessions: LeetCode medium/hard under timed conditions. Verbalize your thinking throughout.
- 5 ML theory sessions: get someone to ask you to derive things from scratch on a whiteboard.
- 5 system design sessions: full 45-minute sessions designing occupancy net, data engine, inference stack.
- Record all sessions. Watch the recording. Note where you hesitate or lose clarity.
- Get feedback specifically on: (1) structure of your answer, (2) depth of technical knowledge, (3) communication clarity.

Resources: *interviewing.io* • *Pramp* • *Exponent ML design course*

Networking, referrals & application strategy

A referral from a current Tesla AI engineer meaningfully shifts your odds. Treat this as a deliverable.

- Identify 10–15 Tesla AI engineers on LinkedIn whose work aligns with your target role.
- Engage authentically: comment thoughtfully on their papers or posts before sending a connection request.
- When reaching out: reference a specific paper or project of theirs. Ask one specific question. Keep it short.
- Attend NeurIPS, CVPR, ICCV, or ICRA (or watch recordings and engage on social): Tesla AI team presents there.
- Apply to all 5 roles simultaneously — they share a candidate pool and a strong signal in one can transfer.

4. Deep Interview Preparation

Tesla AI interviews are technically demanding. Expect questions that require you to derive things from scratch, implement on a whiteboard, and defend design decisions under pressure. The sections below cover every category you are likely to encounter across the five roles, with hints for the harder questions.

Tesla-specific signal to demonstrate in every answer: They care about engineers who own a full ML vertical — data collection strategy, labeling, architecture design, training at scale, evaluation, and production deployment. Frame every answer with this end-to-end ownership mindset.

4.1 Deep Learning Theory

Expect derivation questions. Tesla interviewers frequently ask you to implement things on a whiteboard and prove correctness.

Backpropagation & Gradients

- **Derive the backpropagation equations for a batch normalization layer, including the gradient w.r.t. x, gamma, and beta.**

Hint: Use the chain rule through the normalization step. The gradient w.r.t. x is non-trivial because every element in the batch affects the mean and variance.

- **Why does the vanishing gradient problem occur in deep networks and how does each of the following solve it: residual connections, layer norm, and gating mechanisms?**

Hint: Residual connections provide gradient highways. Layer norm prevents activation scale explosion. Gating (LSTM/GRU) controls gradient flow with learned gates.

- **Derive the gradient of the softmax-cross-entropy combination. Why is computing them separately numerically unstable?**

Hint: Separate computation involves division by a small softmax denominator before taking log. Combined, the softmax terms cancel cleanly, leaving p_i - y_i.

- **What is the straight-through estimator? When is it used and why does it work despite not being a true gradient?**

Hint: Used for quantization and binary activations. Passes gradient through the discontinuity as if it were the identity. Works empirically because the true gradient is zero almost everywhere, which is worse.

- **Explain gradient clipping: why is it needed, and what is the difference between gradient norm clipping and gradient value clipping?**

Hint: Norm clipping rescales the entire gradient vector when its L2 norm exceeds a threshold — preserves direction. Value clipping clips each element independently — does not preserve direction.

Optimizers & Training Dynamics

- **Why does AdamW outperform Adam for training large Transformer models? Derive the weight update equations for both.**

Hint: Adam applies weight decay as an additive term before the adaptive scaling, which means heavily updated weights get less regularization. AdamW decouples weight decay, applying it directly to the parameter — scale-invariant regularization.

- **What is the warmup phase in learning rate schedules, and why is it critical for training large models with Adam?**

Hint: At initialization, the second-moment estimate is near zero, making the adaptive learning rate very large. Warmup allows the running statistics to accumulate before taking large steps.

- **Explain the difference between loss landscape sharpness and generalization. How does SAM (Sharpness-Aware Minimization) address this?**

Hint: Flat minima generalize better because small perturbations in weights produce small changes in loss. SAM seeks parameters where the maximum loss in a neighborhood is minimized — seeks flat regions.

- **What is gradient accumulation and when would you use it? What are its limitations?**

Hint: Simulate large batch sizes by accumulating gradients over multiple forward passes before stepping. Limitation: does not replicate the normalization behavior of true large batches (e.g., batch norm statistics).

- Explain learning rate decay strategies: cosine annealing, polynomial decay, and one-cycle policy. When would you choose each?

Loss Functions & Regularization

- **Why does focal loss outperform cross-entropy for object detection? Derive the focal modulation factor and explain its effect on gradient magnitude.**

Hint: CE treats all samples equally. Focal loss down-weights easy negatives (high confidence correct predictions) via (1-p)^gamma. This concentrates learning on hard examples — critical for dense detection where background hugely outnumbers foreground.

- **Explain contrastive loss (NT-Xent) vs triplet loss vs InfoNCE. What are their tradeoffs in terms of sample efficiency and hyperparameter sensitivity?**

Hint: Triplet loss operates on 3 samples with a margin — unstable mining. NT-Xent and InfoNCE use all negative pairs in a batch — more signal per step. InfoNCE is a lower bound on mutual information.

- **What is label smoothing and why does it improve calibration? How does it affect the optimal logit values a model converges to?**

*Hint: With label smoothing epsilon, the model is penalized for being too confident. The optimal logit difference between correct and incorrect classes is log((1-eps)/eps) * (K-1) rather than infinity.*

- Derive the KL divergence loss used in VAEs. Why does posterior collapse occur and what are the standard mitigations?
- What is the difference between L1 and L2 regression losses in terms of gradient behavior near zero and outlier sensitivity?

Architecture & Attention Mechanisms

- **Explain how Flash Attention achieves sub-quadratic memory usage. Trace through one tile computation, specifying which data lives in SRAM vs HBM at each step.**

Hint: Standard attention materializes the full N×N attention matrix in HBM. FlashAttention tiles the computation into blocks that fit in SRAM, computes running softmax normalization without materializing the full matrix, and only writes O to HBM.

- **What is the difference between Layer Norm and RMS Norm? Why do modern LLMs (LLaMA, Mistral) prefer RMS Norm?**

Hint: RMS Norm drops the mean subtraction step — only scales by the RMS. ~10–15% faster, no meaningful accuracy drop. The centering step is redundant given subsequent linear layers.

- **Explain Multi-Query Attention (MQA) and Grouped-Query Attention (GQA). What problem do they solve and what is the accuracy tradeoff?**

Hint: Standard MHA has K/V heads = Q heads. MQA uses 1 K/V head shared across all Q heads — reduces KV cache by n_heads×. GQA groups Q heads and shares K/V within groups — a middle ground. Reduces memory bandwidth for KV cache loading during inference.

- **How does Rotary Position Embedding (RoPE) encode position? Why does it generalize better to longer sequences than learned absolute positions?**

Hint: RoPE rotates Q and K vectors by position-dependent angles before dot product. The relative position only enters via the angle difference between Q and K positions — the model sees relative distance regardless of absolute position.

- Explain sparse attention patterns (Longformer, BigBird). When would you choose them over Flash Attention for long sequences?
- What are the key architectural differences between a decoder-only transformer (GPT-style) and an encoder-decoder (T5-style)? When would you choose each?

4.2 Computer Vision

Expect questions at the intersection of math and system design. You may be asked to implement a geometric vision algorithm or explain a neural architecture at the operator level.

3D Geometry & Camera Models

- **Derive the projection of a 3D point X into a 2D image pixel u given camera intrinsics K and extrinsics [R|t].**

*Hint: X_cam = R*X + t. X_img = K * X_cam / X_cam[2]. Result: u = (K[0,0]*x/z + K[0,2], K[1,1]*y/z + K[1,2]).*

- **Given two corresponding points in two images and the camera intrinsics, how do you compute the relative pose between the two cameras?**

*Hint: Compute the essential matrix E = K2^T * F * K1 via 5-point or 8-point algorithm. Decompose E = U*diag(1,1,0)*V^T via SVD to get 4 candidate [R|t] pairs. Use cheirality to select the correct one.*

- Explain the difference between the Essential matrix and the Fundamental matrix. When would you use each?
- **How does depth estimation work in a monocular setting? Why is the estimated depth scale-ambiguous and how can scale be recovered?**

Hint: Monocular depth models minimize photometric reprojection error between frame pairs — the ratio D/t is undetermined. Scale can be recovered with: (1) sparse lidar, (2) known object sizes, (3) IMU integration, or (4) stereo training signal.

- Explain the pinhole camera distortion model. What are radial vs tangential distortion coefficients and when does each dominate?
- **How would you calibrate the extrinsic parameters between 8 cameras in a surround-view camera rig like Tesla's?**

Hint: Use a calibration target visible in overlapping camera pairs. Estimate pair-wise extrinsics, then compose along a spanning tree, then refine all cameras jointly with bundle adjustment.

Neural Perception Models

- **Explain the bipartite matching loss in DETR. Why does it avoid duplicate detections without NMS and what is its computational cost at training time?**

Hint: Hungarian algorithm finds the minimum-cost 1-to-1 assignment between predicted and ground-truth boxes. O(n^3) with n=100 predictions — acceptable. No NMS because there is exactly one prediction assigned to each GT — structural deduplication.

- **Compare BEV (bird's-eye view) perception approaches: LSS (Lift-Splat-Shoot), BEVFormer, and BEVFusion. What are the tradeoffs?**

Hint: LSS: predicts depth distribution per pixel, splats features to 3D grid, pools to BEV. Fast but noisy depth. BEVFormer: cross-attention from BEV queries to image features at lifted positions — no explicit depth. BEVFusion: fuse camera BEV + LiDAR BEV in a shared voxel space.

- **Design a camera-only occupancy network for FSD. Describe the data representation, architecture, loss function, and evaluation metrics.**

Hint: Input: N surround cameras. Architecture: backbone + BEV feature lifting (LSS) + 3D voxel decoder. Loss: per-voxel binary cross-entropy + semantic CE. Eval: voxel IoU per class, mIoU, ray-casting distance error.

- How does SAM (Segment Anything Model) achieve class-agnostic segmentation? Explain the promptable design and ambiguity handling.
- Explain the panoptic segmentation task. How does Mask2Former unify instance and semantic segmentation in a single architecture?
- **How would you handle a domain gap between daytime training data and nighttime deployment? What are your options both at training time and at test time?**

Hint: Training: add synthetic nighttime data (style transfer, ControlNet conditioning); use domain randomization; train with domain-adversarial loss. Test time: histogram equalization, retinex enhancement, test-time adaptation (TTT) with a consistency loss.

NeRF, 3DGS & Neural Rendering

- **Derive the volume rendering integral used in NeRF. What is the meaning of the transmittance term and how is it approximated with quadrature?**

*Hint: C(r) = integral[t_n to t_f] T(t)*sigma(r(t))*c(r(t),d) dt. T(t) = exp(-integral[t_n to t] sigma(s)ds). Quadrature: stratified sampling in N bins, alpha compositing with alpha_i = 1 - exp(-sigma_i * delta_i).*

- **What is the key innovation of Instant-NGP over vanilla NeRF? Explain the multiresolution hash encoding.**

Hint: Replace positional encoding with a multiresolution hash table. Each resolution level maps 3D grid corners to feature vectors via a hash function. Collisions are resolved implicitly by the MLP. Result: 1000× speedup, trains in seconds.

- **Explain 3D Gaussian Splatting: what are the learnable parameters of each Gaussian, and how is rasterization performed?**

*Hint: Each Gaussian: position (3D mean), covariance (3×3 decomposed as scale*rotation), opacity, SH color coefficients. Rasterization: project 3D Gaussians to 2D screen-space, sort by depth, alpha-composite front-to-back in tiles.*

- What are the failure modes of NeRF and 3DGS under sparse input views? How does RegNeRF or DNGaussian address this?
- How would you use NeRF or 3DGS to augment a self-driving training dataset with novel viewpoints and rare scenarios?

4.3 ML Inference Optimization

The inference optimization role is the most systems-heavy of the five. Expect questions that combine hardware knowledge with ML mathematics.

Quantization

- **Explain post-training quantization (PTQ) vs quantization-aware training (QAT). When would you choose each and what determines the accuracy degradation?**

Hint: PTQ: no retraining, clips weights/activations to INT range using calibration data. Fast but lossy for aggressive quantization. QAT: inserts fake-quantization nodes in forward pass, uses straight-through estimator in backward. Can recover most accuracy but requires training time.

- **What is the GPTQ algorithm for LLM weight-only quantization? Why does it outperform naive round-to-nearest at INT4?**

Hint: GPTQ uses a second-order approximation (Hessian-based) to compensate for quantization error in already-quantized weights when quantizing the next weight. Processes weights column-by-column within each linear layer, updating remaining weights to minimize output error.

- **Explain the difference between weight-only quantization and weight+activation quantization. Why is activation quantization harder?**

Hint: Activations have dynamic ranges that vary per input (especially attention scores and residuals). Weight ranges are static and calibratable offline. INT4 weight + FP16 activation is a common practical sweet spot.

- What is per-tensor vs per-channel quantization? When does per-channel significantly outperform per-tensor?
- **How would you quantize a model for deployment on Tesla's Full Self-Driving chip (HW4)?**

Hint: Profile the model to identify bottlenecks. Apply INT8 per-channel QAT for convolutional and linear layers. Keep attention softmax in FP16 (dynamic range). Use INT4 for large weight matrices if bandwidth-bound. Validate with chip-level simulator.

Pruning & Distillation

- **Compare structured vs unstructured pruning for on-vehicle inference. Which gives better actual speedup on hardware and why?**

Hint: Unstructured pruning creates sparse weight matrices — requires specialized sparse hardware (not present on most NPUs). Structured pruning removes entire channels, heads, or layers — produces dense smaller tensors, directly maps to standard GEMM operations.

- **Explain the SparseGPT algorithm for one-shot pruning of large language models.**

Hint: SparseGPT processes layers sequentially. For each layer, it uses a second-order approximation (OBS framework) to compute the optimal weight update that compensates for pruning a subset of weights. Achieves 50% sparsity with minimal perplexity increase.

- **Describe a knowledge distillation pipeline for compressing a large FSD perception model to a smaller on-vehicle model. What loss terms would you include?**

Hint: Student mimics teacher at multiple levels: (1) soft label KL divergence (output), (2) intermediate feature distillation (CRD/FitNets style), (3) attention map distillation. Often use a temperature T>1 for soft labels to soften the distribution.

- What is head importance scoring in attention pruning? How would you determine which attention heads to prune in a ViT?
- Explain lottery ticket hypothesis. Is it practical for production model compression? What are its limitations?

Hardware & Systems

- **Explain the roofline model. Given a model's arithmetic intensity, how do you determine whether it is compute-bound or memory-bandwidth-bound on a specific GPU?**

Hint: Arithmetic intensity = FLOPs / bytes of memory accessed. If intensity > GPU peak FLOPS / peak memory bandwidth (the ridge point), it is compute-bound. Otherwise, memory-bandwidth-bound. Most LLM inference with small batch sizes is memory-bandwidth-bound.

- **Why is batch size 1 autoregressive LLM inference almost always memory-bandwidth-bound? How do continuous batching and speculative decoding address this?**

Hint: At batch=1, the GPU loads all KV cache and model weights for a single token generation — essentially serving a single GEMV per step. Continuous batching amortizes weight loads across concurrent requests. Speculative decoding generates multiple draft tokens with a small model, verified in parallel by the large model.

- **Explain the CUDA memory hierarchy and why coalesced memory access matters for kernel performance.**

Hint: Register → shared memory (SRAM, ~100 TB/s) → L2 → HBM (~3 TB/s). Coalesced access: threads in a warp access consecutive memory addresses → served in a single transaction. Non-coalesced: multiple transactions, proportional slowdown.

- What is CUDA graph capture and why does it reduce inference latency for static computation graphs?
- **How would you profile a newly quantized detection model to verify it is running efficiently on an NPU?**

Hint: Use hardware-specific profiler (e.g., NSight for NVIDIA, Snapdragon Profiler for mobile NPUs). Check: (1) layer-wise latency breakdown, (2) memory bandwidth utilization vs peak, (3) compute utilization vs peak, (4) stall breakdown (memory vs pipeline vs synchronization).

- Explain KV cache memory management strategies: paged attention, prefix caching, and multi-query attention. How do they interact?

4.4 Data Scaling & Data Engine Design

The data scaling role is about system-level thinking applied to ML data. Expect questions that blend ML mathematics with data pipeline engineering.

Data Curation & Quality

- **How would you design a data engine to surface hard negatives from Tesla's fleet of 10 million vehicles? Walk through the full pipeline.**

Hint: (1) Trigger collection: uncertainty-based (entropy of softmax output), disagreement-based (ensemble divergence), or failure-case mining (post-incident reporting). (2) Deduplication: locality-sensitive hashing on image embeddings. (3) Labeling: human-in-the-loop for rare cases, automated for common. (4) Curriculum: schedule hard examples at the right point in training.

- **What is curriculum learning? When does it help and when does it hurt?**

Hint: Train on easy examples first, progressively introduce harder ones. Helps when hard examples early cause optimization instability. Can hurt if the model overfits to easy examples and struggles to transfer to hard distribution. Alternative: hard example mining without ordering (online hard example mining, OHEM).

- Explain active learning. What are the query strategies (uncertainty, diversity, core-set) and their tradeoffs at fleet scale?
- **How do you determine the optimal data mixture for multi-task pre-training across driving scenarios (highway, city, rain, night)?**

Hint: Start with the Chinchilla framework — equal token distribution as baseline. Measure downstream task performance sensitivity to each source. Use DoReMi or data influence functions to estimate the optimal mixture weights. Iterate: collect more of the high-value sources.

- What is dataset deduplication and why does it matter for model training? Describe a scalable deduplication pipeline for image data.

Evaluation Pipeline Design

- **How would you design an evaluation pipeline for Tesla's FSD driving policy that surfaces rare failure scenarios, not just average-case performance?**

Hint: (1) Slice-based eval: per-scenario (rain, pedestrian crossing, construction), per-region, per-time-of-day. (2) Long-tail mining: embed scenarios, cluster, find sparse clusters (rare scenarios). (3) Adversarial eval: generate near-failure inputs with gradient-based or evolutionary search. (4) Human expert red-teaming.

- What is the difference between online (in-distribution) and offline (held-out) evaluation for learned driving policies? When does each mislead you?
- **How would you design a metric for autonomous driving that correlates with real-world safety, given that you cannot always measure ground-truth outcomes?**

Hint: Proxy metrics: (1) intervention rate (human takeovers per mile), (2) minimum time-to-collision (TTC), (3) comfort metrics (jerk, acceleration), (4) trajectory deviation from expert driver. Validate by correlating proxy metrics with labeled safety outcomes on a held-out set.

- Explain data flywheel design for self-driving: how does each deployment of the model generate better training data for the next version?
- How would you measure and mitigate distribution shift between your training data and your deployment distribution?

4.5 ML System Design

Full 45-minute system design questions. Practice these end-to-end. You will be evaluated on: problem decomposition, data thinking, architecture choices, evaluation strategy, and production considerations.

Full System Design Problems

- **Design Tesla's camera-only BEV occupancy prediction system from scratch.**

Hint: Data: nuScenes + synthetic. Architecture: multi-camera backbone (shared) → BEV feature lifting (LSS) → 3D voxel decoder. Training: voxel-level CE + lovász loss. Eval: voxel IoU per class. Deployment: INT8 QAT, TensorRT export, latency target 10ms on HW4. Online eval: compare predicted occupancy to LiDAR ground truth on validation fleet.

- **Design a data engine for automatically mining and labeling rare driving scenarios from the Tesla fleet.**

Hint: Trigger: uncertainty-based (softmax entropy) + failure-case (accident-adjacent telemetry). Dedup: image embedding + LSH. Prioritization: expected model improvement (EL2N scores). Labeling: automated for common cases, human-in-loop for rare. Feedback loop: deploy model → mine failures → label → retrain → deploy.

- **Design the ML inference stack for deploying a 3B-parameter multimodal driving model on Tesla's HW4 chip.**

Hint: Quantize: INT8 weight+activation QAT for convolutional layers, INT4 weight-only for attention. Prune: structured head pruning (30% heads removed). Distill: student matches teacher's BEV feature maps. Export: ONNX → TensorRT with custom plugins for fused ops. Validate: accuracy within 1% mAP of FP32 teacher. Latency: <20ms per frame on HW4.

- **Design a multimodal pretraining pipeline for a vision-language foundation model for Tesla's robots (Optimus).**

Hint: Data: internet video + text, robot teleoperation data, simulated interaction data. Architecture: ViT backbone + resampler (compress 256 tokens to 64) + LLM decoder. Training stages: (1) image-text contrastive (CLIP-style), (2) generative VLM pretraining, (3) robot action prediction finetuning. Eval: zero-shot task completion in simulation, then hardware.

- **Design a system that uses a world model to generate synthetic driving data for rare weather conditions (heavy snow, dense fog).**

Hint: World model: finetune a video diffusion model (e.g., Stable Video Diffusion) conditioned on driving scenario + weather text prompt. Validation: FID between synthetic and real distribution. Training: augment dataset with synthetic samples. Measure: mAP improvement on real rare-weather held-out set after augmentation.

5. Key Resources at a Glance

Category	Resource	Why it matters
DL fundamentals	Neural Nets: Zero to Hero (Karpathy)	Best hands-on intro to modern DL. Build everything from scratch.
DL fundamentals	Lillian Weng's blog (lilianweng.github.io)	Deep, accurate survey posts on every major DL topic.
DL fundamentals	The Little Book of Deep Learning (Fleuret)	Concise, mathematically precise. Perfect for theory review.
Foundation models	CLIP, DINO, DINOv2, MAE papers	Core SSL methods that underpin Tesla's pretraining. Must know cold.
Foundation models	LLaVA / PaliGemma / InternVL papers	Multimodal VLM architecture patterns.
Foundation models	Chinchilla + Scaling Laws papers	Scaling theory that drives data and compute decisions.
3D vision	Multiple View Geometry (Hartley & Zisserman)	The canonical 3D CV textbook. Chapters 9–12 especially.
3D vision	NeRF paper + Instant-NGP + nerf.studio	Neural rendering from foundations to fast implementation.
3D vision	3D Gaussian Splatting paper	Modern real-time radiance fields.
3D vision	BEVFusion / LSS / BEVFormer papers	Camera-to-BEV lifting methods used in FSD-style systems.
Inference opt.	FlashAttention 1 & 2 papers	Memory-efficient attention — deep hardware knowledge.
Inference opt.	GPTQ / SparseGPT / AWQ papers	State-of-art LLM compression methods.
Inference opt.	OpenAI Triton tutorials	Custom GPU kernels in Python.
Inference opt.	CUDA by Example (Sanders & Kandrot)	Foundational CUDA programming.
Robotics	Diffusion Policy / ACT papers	Modern imitation learning for manipulation.
Robotics	GAIA-1 / DriveDreamer papers	World models for generative driving data.
Tesla-specific	Tesla AI Day 2021 (YouTube)	Occupancy nets, vector space, data pipeline at scale.
Tesla-specific	Tesla AI Day 2022 (YouTube)	Dojo supercomputer, ExaPOD, virtual environments.
Coding prep	LeetCode (Neetcode 150 list)	Structured coverage of all coding interview patterns.
Interview prep	interviewing.io / Pramp	Live mock interviews with experienced engineers.
System design	Designing ML Systems (Chip Huyen)	Best book on production ML thinking.

Own the full stack. Good luck.